

Desempenho de inteligências artificiais na criação de APIs para plataformas de simulação de investimentos.

Igor Ayello Borges^{1*}; Anderson Canale Garcia²

¹ Aluno. Rua Carneiro Leão – Vila Scarpelli; 09050-430 Santo André, São Paulo, Brasil

² Mestre em Ciência da Computação. Avenida Trabalhador São-carlense, 400 - Centro; 13560-970 São Carlos,
São Paulo, Brasil

*autor correspondente: igor8n1@gmail.com

Desempenho de inteligências artificiais na criação de APIs para plataformas de simulação de investimentos.

Resumo

Com o avanço da inteligência artificial diversas ferramentas têm sido utilizadas para automatizar tarefas e aumentar a produtividade no desenvolvimento de software, incluindo a criação de APIs, porém não existe definição sobre o desempenho entre esses modelos em tarefas específicas. Este trabalho tem como objetivo comparar o desempenho de cinco modelos de IA (Copilot, ChatGPT, DeepSeek, Gemini e Meta AI) na geração de APIs para um simulador de investimentos, avaliando pontos como qualidade do código, segurança e eficiência estrutural (manutenibilidade e complexidade do código, conforme métricas do SonarQube). A pesquisa adotou uma abordagem experimental quantitativa, utilizando tecnologias como Python, FastAPI, Docker e SonarQube para análise técnica. Os dados financeiros utilizados como base para os testes foram obtidos de APIs externas. Os resultados apontam diferenças entre os modelos em aspectos como segurança e eficiência, permitindo considerar quais ferramentas apresentam melhor desempenho para esse tipo de aplicação.

Palavras-chave: Inteligência artificial; Comparação AI; Desempenho SonarQube; Chatbot

Introdução

O avanço das tecnologias de extração, armazenamento, transmissão e processamento de dados tem possibilitado avanços no desenvolvimento da Inteligência Artificial (IA) nos últimos anos (Carvalho, 2021). Esse progresso permitiu a criação de novas soluções e produtos, transformando os processos de desenvolvimento. Brandão (2020) destaca que a integração de tecnologias de IA nos processos organizacionais tem resultado em um melhor atendimento às demandas dos clientes, demonstrando o impacto positivo da adoção dessas ferramentas. Aplicações como o ChatGPT e o GitHub Copilot têm mostrado capacidade de auxiliar desenvolvedores em tarefas específicas, como a geração de código (Ignácio et al., 2024). No entanto, desafios relacionados à precisão e confiabilidade ainda são enfrentados por essas ferramentas, limitando sua aplicabilidade em projetos mais complexos.

Segundo o DataReportal (2025), plataforma que analisa dados digitais globais e fornece informações sobre tendências, entre as ferramentas de inteligência artificial mais populares em 2025 estão o Copilot, ChatGPT, DeepSeek, Gemini e Meta AI, cada uma apresentando características distintas e modelos específicos. O Copilot e o ChatGPT, por exemplo, utilizam modelos desenvolvidos pela OpenAI, enquanto o DeepSeek emprega arquiteturas próprias. O Gemini e o Meta AI integram os ecossistemas do Google e da Meta, respectivamente. A capacidade dessas ferramentas de integrar processos e otimizar o trabalho de desenvolvedores tem sido destacada (Peng et al., 2023).

Nesse contexto, o uso da IA no desenvolvimento de software tem sido intensificado com o objetivo de aumentar a produtividade e melhorar a qualidade dos códigos e produtos gerados (Cui et al., 2025). No entanto, uma lacuna permanece na avaliação da eficácia de diferentes modelos de inteligência artificial na criação de APIs confiáveis e funcionais (Wu et

al., 2024). Dado o crescimento da aplicação dessas tecnologias no desenvolvimento de software, torna-se essencial compreender de que forma as ferramentas disponíveis podem contribuir para a criação de aplicações específicas, identificando limitações, oportunidades de melhoria, pontos fortes e desafios.

Diante disso, este estudo propõe uma abordagem comparativa do desempenho de cinco modelos de inteligência artificial na geração de APIs para um simulador de investimentos. Ao término da pesquisa, espera-se que métricas e tendências sejam identificadas, contribuindo para o aprimoramento do uso da IA no desenvolvimento de software.

Metodologia

De acordo com Lakatos e Marconi (2025), a pesquisa experimental é caracterizada pelo controle de variáveis e pela observação de resultados quando se busca verificar relações de causa e efeito em ambientes controlados. Neste trabalho, adotou-se uma abordagem experimental e quantitativa, com o objetivo de comparar o desempenho de diferentes modelos de inteligência artificial na criação de APIs voltadas à simulação de investimentos. A pesquisa foi conduzida utilizando ferramentas tecnológicas específicas e métricas padronizadas para uma análise comparativa.

O delineamento metodológico consistiu na utilização de cinco modelos de inteligência artificial utilizados em 2025 sendo: ChatGPT (versão 3.5), Copilot (o3-mini), DeepSeek (V3 0324), Gemini (2.5 Flash) e Meta AI (Llama 3). Para cada modelo, foi solicitado gerar uma versão backend de uma API implementando funcionalidades como importação e análise de dados financeiros, consultas a APIs externas, cálculos de risco e retorno, armazenando informações em banco de dados relacional. Todas as inteligências artificiais foram acessadas por meio "chat", visando padronizar o processo para garantir a comparação dos resultados já que cada modelo recebeu os mesmos prompts. Não foram utilizadas integrações alternativas como o uso de IDEs ou o consumo via API evitando que funcionalidades adicionais influenciassem o resultado da análise.

Para comparar os resultados definiram-se as seguintes variáveis:

Variável independente: inteligências artificiais usadas (Copilot, ChatGPT, DeepSeek, Gemini e Meta AI);

Variáveis dependentes: métricas extraídas do SonarQube sendo: número de bugs, vulnerabilidades, code smells, complexidade do código, tempo estimado de correção e índice de dívida técnica;

Variáveis de controle: linguagem de programação (Python¹), framework (FastAPI²), virtualização (Docker³), banco de dados (SQL Server⁴) e prompts idênticos aplicados a todos os modelos utilizando arquitetura de software Clean⁵.

A coleta de dados financeiros foi realizada a partir de APIs externas como o YFinance (para séries históricas do Ibovespa) e a API do Banco Central (para índices do CDI diário). Os dados foram armazenados em tabelas no SQL Server, servindo de base para a implementação e execução dos experimentos.

As APIs geradas pelos modelos de inteligência artificial foram avaliadas no SonarQube, que forneceu métricas objetivas relacionadas à qualidade, segurança e eficiência estrutural do código (facilidade de manutenção, simplicidade estrutural e menor dívida técnica, medidos por indicadores como complexidade ciclomática e cognitiva, tempo estimado de manutenção e índice de dívida técnica). A análise quantitativa dos resultados foi conduzida por meio de tabelas comparativas e gráficos, permitindo identificar padrões de desempenho, vantagens e limitações de cada modelo.

Os detalhes técnicos, incluindo a lista completa de prompts utilizados estão documentados no Apêndice A para manter a seção metodológica focada no delineamento experimental e na fundamentação científica.

As datas de desenvolvimento foram as seguintes: ChatGPT versão 3.5 (5/03/2025 a 15/03/2025); Copilot versão o3-mini (23/03/2025 a 26/03/2025); DeepSeek versão V3 0324 (29/03/2025 a 31/03/2025); Gemini versão 2.5 Flash (03/04/2025 a 12/04/2025); MetaAI versão Llama 3 (14/04/2025 a 16/04/2025). A avaliação no SonarQube versão 2025.4 foi realizada em 30/04/2025.

Resultados e Discussão

Todos os requisitos foram cumpridos pelas cinco inteligências artificiais, resultando na criação de APIs funcionais. As diferenças apresentadas nas tabelas e análises não indicam falha ou incapacidade de alguma IA, mas variações no grau de qualidade, manutenção, segurança e esforço de manutenção dos códigos gerados. Todas as perguntas feitas no prompt exigiram explicações sobre a linguagem de programação utilizada, Python, o framework FastAPI e a arquitetura de software Clean.

¹ <https://www.python.org/>

² <https://fastapi.tiangolo.com/>

³ <https://www.docker.com/>

⁴ <https://www.microsoft.com/pt-br/sql-server/sql-server-downloads>

⁵ <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>

As principais dificuldades encontradas para tornar o código funcional estiveram relacionadas à integração entre containers no Docker, à criação de arquivos para debug no VSCode e, no que diz respeito ao código, ao acesso às APIs do Banco Central e do Yahoo Finance, que foi realizado de maneira satisfatória, exigindo apenas pequenos ajustes. As maiores dificuldades foram observadas nos cálculos do Beta, índice Sharpe, volatilidade e percentil, exigindo ajustes nos códigos para garantir a obtenção dos valores corretos, esses ajustes foram realizados em todas as IAs para que o resultado final fosse realizado de maneira uniforme, no entanto talvez mesmo os ajustes aplicados de forma uniforme a todos os modelos, é possível que a necessidade de maior ou menor intervenção em determinados códigos tenha influenciado o desempenho final medido pelo SonarQube, como nos indicadores de eficiência estrutural (complexidade, dívida técnica e tempo de manutenção). Considerando a integração com APIs externas como Banco Central e Yahoo Finance foi decidido implementar por meio do “design pattern” “adapter” (adaptador) utilizado para converter o retorno de uma API que o sistema não consegue ler para outro retorno que o sistema espera ler sem alterar o código original, todas as IAs tiveram dificuldade em criar o padrão de código sendo assim decidi fazer a implementação manualmente sem auxílio da IA. Por fim quando foi solicitado para todas as IAs que fizessem operações tanto de leitura quanto de escrita no banco de dados todas sugeriram o uso da ferramenta SQLAlchemy, biblioteca de mapeamento objeto-relacional, porém como as operações no banco movimentam muitos dados optei por realizar essas operações de maneira nativa no SQL e para isso precisei especificar para cada uma das IAs que necessito de operações nativas.

No SonarQube, foram realizados testes abrangentes, incluindo a verificação de vulnerabilidades, “bugs”, “code smells” (padrões de código), “accepted issues” (problemas conhecidos, porém aceitos que podem ser definidos como falso positivo ou por decisão consciente do usuário de não corrigir ou corrigir no futuro), duplicações, “security hotspots” (trechos de código que podem representar riscos à segurança mas que exigem análise manual do desenvolvedor), severidade dos “bugs”, tempo de manutenção, manutenibilidade e complexidade. A seguir, serão apresentados os resultados detalhados.

Visão geral

Esta seção apresentará a avaliação geral das cinco inteligências artificiais segundo o SonarQube. A análise teve início pela verificação de vulnerabilidades, cujo objetivo é identificar possíveis falhas de segurança nos códigos gerados conforme verificado na Tabela 1.

Tabela 1. Segurança

Inteligência artificial	Número de vulnerabilidades	Avaliação vulnerabilidade	Bug	Avaliação bugs	Code Smells	Avaliação Code Smells
ChatGPT	1	E	1	C	20	A
Copilot	1	E	0	A	8	A
DeepSeek	1	E	0	A	29	A
Gemini	2	E	0	A	13	A
MetaAI	2	E	0	A	5	A

Fonte: Resultados originais da pesquisa (2025)

Ao avaliar as vulnerabilidades segundo o SonarQube, foi identificado que os modelos Gemini e MetaAI apresentaram duas vulnerabilidades, enquanto ChatGPT, Copilot e DeepSeek registraram apenas uma. Todas as inteligências artificiais receberam a avaliação "E", indicando que há ao menos uma vulnerabilidade bloqueadora, caracterizada como uma falha crítica capaz de comprometer a segurança do software.

O número de “bugs” gerados tem como objetivo identificar possíveis falhas no código, que podem resultar em comportamento inesperado da aplicação, comprometimento da segurança e dificuldade na manutenção. Com exceção do ChatGPT, nenhum “bug” foi gerado pelas demais inteligências artificiais, todas recebendo a avaliação A (zero “bugs”). O único bug identificado no código gerado pelo ChatGPT recebeu a avaliação "C", indicando a presença de falhas na confiabilidade, embora sem impacto na execução.

Considerando a métrica “Code Smells”, que avalia possíveis problemas relacionados à manutenção e qualidade. Embora esses códigos não impeçam a execução do software, podem dificultar o processo de manutenção, todas as inteligências artificiais receberam avaliação “A”, indicando que os códigos gerados estão bem estruturados e não apresentam problemas significativos de manutenção. Em relação à quantidade de “code smells”, a Meta AI apresentou o menor número (5), enquanto a DeepSeek registrou a maior quantidade (29). Isso sugere que o código gerado pela DeepSeek pode demandar mais ajustes ao longo do tempo em comparação com o da Meta AI, resultando em maior esforço de manutenção para os desenvolvedores, conforme será analisado nas avaliações futuras.

Nas métricas referentes à “accepted issues” (problemas reconhecidos, porém minoritários), duplicações e “security hotspots” (trechos de código que podem representar riscos à segurança mas que exigem análise manual do desenvolvedor), todas as inteligências artificiais apresentaram resultados satisfatórios, com zero ocorrências. A avaliação de

segurança foi classificada como “A”, indicando que não foram identificados riscos significativos para a segurança da informação.

O próximo passo avaliado foi a análise de severidade do código identificando o quão grave é cada severidade sendo: “Blocker”: Problemas críticos que podem causar falhas graves no software e serão priorizados no momento da correção. “Critical”: Problemas que representam riscos como, por exemplo, vulnerabilidades de segurança ou erros que comprometem a execução do software. “Major”: Problemas referentes a qualidade do código e podem diminuir a produtividade dos desenvolvedores. “Minor”: Problemas que não afetam o funcionamento do código, porém, podem ser melhorados. “Info”: Sugestões de melhoria. A partir dessas definições tive os seguintes resultados observados na Tabela 2:

Tabela 2. Severidades

Inteligência artificial	Total	Bloqueios	Críticas	Principais	Secundárias
ChatGPT	22	1	0	9	12
Copilot	9	1	3	3	2
DeepSeek	30	1	9	13	7
Gemini	15	2	1	9	3
MetaAI	7	2	0	4	1

Fonte: Resultados originais da pesquisa (2025)

Nenhuma das cinco inteligências artificiais apresentou severidade do tipo “info”.

Na análise das severidades, o menor número de ocorrências foi registrado pela MetaAI, enquanto a DeepSeek apresentou a maior quantidade, com mais de quatro vezes o volume identificado na MetaAI. Apesar de a MetaAI e o Gemini terem gerado menos severidades, ambos registraram duas ocorrências cada em comparação às demais inteligências artificiais.

Em relação às severidades críticas, nenhuma ocorrência foi identificada na MetaAI e no ChatGPT, enquanto a DeepSeek apresentou nove registros. Quanto às severidades principais, o Copilot foi a ferramenta com o menor número de ocorrências, totalizando três, enquanto a DeepSeek apresentou a maior quantidade, com treze registros.

Nas severidades secundárias, a menor quantidade foi registrada pela MetaAI, com apenas uma ocorrência, enquanto o ChatGPT apresentou doze.

O próximo aspecto analisado será o tempo estimado para a correção das severidades, considerando o tempo total, as falhas de segurança, a confiabilidade e a manutenção

Tabela 3. Tempo

Inteligência artificial	Total	Segurança	Confiabilidade	Manutenção
ChatGPT	02h27	00h30	00h10	01h47
Copilot	01h14	00h30	00h00	00h44
DeepSeek	04h30	00h30	00h00	04h00
Gemini	03h45	01h00	00h00	02h45
MetaAI	02h07	01h00	00h00	01h07

Fonte: Resultados originais da pesquisa (2025)

O menor tempo necessário para ajustes no código foi registrado no Copilot, com uma duração de uma hora e quatorze minutos, enquanto o maior tempo foi observado no DeepSeek, totalizando quatro horas e trinta minutos.

Na análise da capacidade de manutenção, o SonarQube apresentou os seguintes dados: índice de dívida técnica (“Debt Ratio”), métrica utilizada para dimensionar a dívida técnica. Quanto maior esse índice, maior será o tempo necessário para a correção de “bugs”. Além disso, foram avaliadas a quantidade de linhas de código, funções, classes e arquivos gerados.

Tabela 4. Índice de dívida técnica

Inteligência artificial	Índice de dívida	Linhas de código	Funções	Classes	Arquivos
ChatGPT	0,4	867	105	22	34
Copilot	0,2	826	94	17	28
DeepSeek	0,6	1272	96	18	29
Gemini	0,5	1017	95	16	29
MetaAI	0,3	799	102	16	28

Fonte: Resultados originais da pesquisa (2025)

O menor número de linhas e classes foi gerado pela MetaAI, enquanto a menor quantidade de funções e arquivos foi registrada pelo Copilot. Embora o ChatGPT tenha ocupado a terceira posição em relação ao total de linhas de código geradas, foi o modelo que apresentou a maior quantidade de funções, classes e arquivos.

Para a conclusão da análise, são considerados os indicadores de complexidade, incluindo a complexidade ciclomática e a complexidade cognitiva. A complexidade ciclomática é uma métrica utilizada para medir a complexidade de um programa com base no número de

caminhos independentes que podem ser percorridos no código. Quanto maior esse índice, mais difícil pode ser a realização de testes, manutenção e depuração do código. Já a complexidade cognitiva é empregada para avaliar a dificuldade de compreensão do código. Um índice elevado nessa métrica pode aumentar o risco de erros e dificultar os processos de revisão e refatoração.

Tabela 5. Complexidade

Inteligência artificial	Ciclomática	Cognitiva
ChatGPT	169	76
Copilot	158	87
DeepSeek	182	116
Gemini	182	144
MetaAI	142	68

Fonte: Resultados originais da pesquisa (2025)

Os códigos menos complexos nas duas categorias foram gerados pela MetaAI, enquanto DeepSeek e Gemini apresentaram maior complexidade em ambas as métricas.

O desenvolvimento das cinco versões encontrou alguns problemas relacionados à otimização de códigos. Dois exemplos podem ser citados. O primeiro diz respeito à consulta de dados: quando solicitado que a consulta fosse feita, todas as IAs sugeriram usar o ORM SQLAlchemy do Python. O uso de ORM pode facilitar a escrita e leitura de dados; porém, em caso de consultas de grande volume – como o histórico da taxa de juros ou o histórico diário dos últimos três anos de uma ação – a API pode apresentar lentidão. Para resolver esse problema, foi necessário recorrer a consultas em SQL nativo. Especificou-se, portanto, para todas as IAs que as consultas deveriam ser feitas dessa forma.

O segundo ponto de atenção foi o acesso a APIs externas, como a de taxa de juros na API do Banco Central e a do Yahoo Finance. Quando foi solicitado às IAs que implementassem um “design pattern” do tipo Adapter, elas criaram apenas um método genérico de consulta, o que dificultaria a manutenção futura desses acessos.

O objetivo da comparação não foi avaliar a capacidade das IAs de gerar APIs funcionais, mas identificar diferenças de qualidade, segurança e eficiência estrutural entre os códigos gerados. Todas as inteligências artificiais cumpriram os requisitos básicos e foram capazes de entregar APIs funcionais.

Considerações Finais

O estudo apresentou algumas limitações que serão consideradas na interpretação dos resultados. Os experimentos foram realizados em um único caso de uso, baseado em APIs desenvolvidas com Python e FastAPI, restringindo a generalização dos resultados para outras tecnologias, além disso a avaliação baseou em métricas fornecidas pelo SonarQube, que não contemplam pontos como desempenho em tempo de execução, consumo de recursos ou experiência do desenvolvedor, outro ponto a destacar é que os modelos de inteligência artificial evoluem rapidamente e estão em constante treinamento sendo que os resultados mostram o estado das ferramentas no momento da pesquisa e podem diferir em análises futuras.

Os resultados mostraram que todas as inteligências artificiais geraram APIs funcionais cumprindo os requisitos definidos. Entretanto, houve diferenças de desempenho em diversos pontos. O Copilot apresentou melhor desempenho pelo menor tempo de correção de "bugs" e ajustes, além de boa estabilidade geral.

A MetaAI apresentou menor índice de complexidade e dívida técnica, além de poucos "code smells", o que sugere que o uso dessa inteligência artificial proporciona maior facilidade de manutenção.

ChatGPT e DeepSeek apresentaram maior número de severidades, principalmente "majors" e "minors", o que indica maior esforço em futuras manutenções.

Gemini e DeepSeek geraram códigos mais complexos, o que pode impactar negativamente os processos de testes e manutenção.

A soma de todos os resultados sugere que o Copilot é a melhor opção para cenários que exigem rapidez no desenvolvimento e menos retrabalho, enquanto a MetaAI é melhor em qualidade estrutural e facilidade de manutenção. De uma forma prática, a IA que se mostrou mais bem-sucedida nos testes foi o Copilot. Ela está integrada a diversas soluções da Microsoft, como as versões recentes do Windows, o Teams e o Github Copilot integrado com o Visual Studio (IDE para desenvolvimento de software da Microsoft). O bom desempenho obtido pelo Copilot não significa que as demais IAs não cumpram o seu papel de facilitar o desenvolvimento de software. Conforme discutido na seção de resultados, todas atenderam aos requisitos mínimos para o funcionamento do software. Deve-se considerar ainda que os algoritmos que treinam as IAs estão em constante evolução, de modo que os resultados obtidos neste estudo podem diferir em análises futuras, conforme o avanço dessas tecnologias.

Os resultados sugerem que o uso de IA no desenvolvimento de software deve ser feito de forma crítica e contextualizada pelo desenvolvedor. Ferramentas como o Copilot são

indicadas quando demanda por agilidade, enquanto modelos como a MetaAI oferecem vantagens em projetos que necessitam código mais limpo e sustentável, além disso independente da IA escolhida o estudo mostra a necessidade de revisão humana para ajustes técnicos e para correções de riscos de segurança. Por fim os resultados alcançaram o objetivo proposto e contribuem para o entendimento de como diferentes inteligências artificiais podem apoiar o desenvolvimento de APIs oferecendo vantagens para que desenvolvedores e empresas façam escolhas sobre quando e como usar essas ferramentas em seus projetos.

O estudo atingiu o objetivo proposto de comparar o desempenho de cinco modelos de inteligência artificial, Copilot, ChatGPT, DeepSeek, Gemini e MetaAI, na criação de APIs para simulação de investimentos. A análise mostrou que embora todos os modelos foram capazes de gerar soluções funcionais houveram diferenças em aspectos de segurança, complexidade e manutenção, permitindo identificar pontos fortes e limitações. Os resultados conectam-se à proposta inicial demonstrando que a comparação entre os modelos foi viável e em consequência contribui para orientar desenvolvedores e organizações na escolha mais adequada de ferramentas de IA para o desenvolvimento de software.

Referências

Brandão, R. 2020. *Inteligência artificial, trabalho e produtividade*. São Paulo, SP, Brasil.

Carvalho, A.C.P.L.F. 2021. *Inteligência artificial: riscos, benefícios e uso responsável*. São Carlos, SP, Brasil.

Cui, K.Z.; Demirer, M.; Jaffe, S.; Musolff, L.; Peng, S.; Salz, T. 2025. *The effects of generative ai on high-skilled work: evidence from three field experiments with software developers*. Philadelphia, PA, Estados Unidos.

DataReportal. 2025. Digital 2025: *Ai making gains*. Disponível em: https://datareportal.com/reports/digital-2025-sub-section-ai-making-gains?utm_source=chatgpt.com. Acesso em: 30/05/2025.

Ignácio, A.C.; Oliveira, L.S.; Francez, M.P.M. 2024. *Eficiência do uso da inteligência artificial no desenvolvimento de software*. Jundiaí, SP, Brasil.

Marconi, M.A.; Lakatos, E.M. 2025. *Técnicas de pesquisa*. 9ed. Atlas, São Paulo, SP, Brasil. Disponível em: <<https://app.minhabiblioteca.com.br/reader/books/9788597026610/epubcfi>>. Acesso em: 18 Set. 2025.

Peng, S.; Kalliamvakou, E.; Cihon, P.; Peter.; Demirer, M. 2023. *The impact of ai on developer productivity: evidence from GitHub Copilot*. Redmond, WA, Estados Unidos.

Wu, Y.; He, P.; Wang, Z.; Wang, S.; Tian, Y.; Chen, T.H.P. 2024. *A Comprehensive Framework for Evaluating API-oriented Code Generation in Large Language Models*. Winnipeg, Canadá.

Apêndice A – Prompts utilizados no desenvolvimento das APIs

A seguir constam os prompts usados para a criação dos códigos utilizados no trabalho, os prompts foram divididos de acordo com a finalidade sendo: informações preliminares, importação de dados sobre as ações, consulta de dados sobre as ações, consumir dados de APIs externas e cálculo de índices.

1- Informações preliminares

Considerando a seguinte arquitetura clean

```
/app
  /core
  /domain
    /models
    /repositories
    /services
  /application
    /interfaces
    /use_cases
  /infrastructure
    /database
    /repositories
  /api
    /v1
  /tests
main.py
requirements.txt
Dockerfile
.env
README.md
```

Fonte: Elaborado pelo autor (2025)

2- Importação de dados sobre as ações

Como criar uma API na FastAPI que leia um arquivo em Excel,
leia os dados da coluna A da linha 521 até a linha 529 e salve na tabela

```
CREATE TABLE dbo.SegmentoClassificacao (
    ID INT IDENTITY(1,1) PRIMARY KEY, --
    Chave primária autoincrementada
    Sigla VARCHAR(10) NOT NULL, --
    Sigla extraída dos parênteses
```

```
                Descritivo VARCHAR(255) NOT NULL    --  
                Texto após os parênteses  
            );
```

sendo que o registro na planilha está da seguinte forma (DR1) BDR Nível 1,
o que está dentro do parenteses DR1 deve ser salvo na coluna Sigla do banco o que
está depois do parenteses) deve ser salvo na coluna descritivo

Como criar um método que leia um arquivo em Excel,
leia os dados da coluna B da linha 9 até a linha 509 e salve na tabela

```
CREATE TABLE dbo.[SubSetor] (  
    ID INT IDENTITY(1,1) PRIMARY KEY,    --  
    Chave primária autoincrementada  
    Descritivo VARCHAR(255) NOT NULL    --  
    Nome do setor econômico  
);
```

sendo que deve-se desconsiderar as células que estejam com o valor SUBSETOR ou
valores em branco

Como criar um método que leia um arquivo em Excel,
leia os dados da coluna A da linha 9 até a linha 509 e salve na tabela

```
CREATE TABLE dbo.[SetorEconomico] (  
    ID INT IDENTITY(1,1) PRIMARY KEY,    --  
    Chave primária autoincrementada  
    Descritivo VARCHAR(255) NOT NULL    --  
    Nome do setor econômico  
);
```

sendo que deve-se desconsiderar as células que estejam com o valor SETOR
ECONÔMICO ou valores em branco

Como criar um método que leia um arquivo em Excel, na coluna C das linhas 8 até
520 sendo que precisa desconsiderar a célula com o valor 'SEGMENTO' e o conteúdo da
coluna D tem que estar vazia

```
CREATE TABLE dbo.Segmento (  
    ID INT IDENTITY(1,1) PRIMARY KEY,    --  
    Chave primária autoincrementada  
    Descritivo VARCHAR(255) NOT NULL    --  
    Nome do segmento  
);
```

Preciso inserir dados na tabela abaixo sendo que os dados vem de uma planilha em Excel e na coluna nome o conteúdo está na coluna C da planilha da linha 8 até a 520 desconsiderando as células com o valor SEGMENTO

e precisa ter algum conteúdo na coluna D, a coluna D da planilha tem o conteúdo que será salvo na coluna Codigo.

Sobre a SegmentoClassificacaoID o conteúdo está na coluna E da planilha porém o valor pode estar vazio, quando não for vazio ela terá a sigla que corresponde a coluna Sigla na tabela dbo.SegmentoClassificacao e a partir

dessa sigla deverá trazer o id correspondente

Sobre a SetorEconomicoID o conteúdo está na coluna A e ele deve ser a primeira célula acima da linha que contenha um valor não nulo e diferente da palavra 'SETOR ECONÔMICO' ela terá o Descritivo que corresponde a coluna Descritivo

na tabela dbo.[SetorEconomico] e a partir desse Descritivo deverá trazer o id correspondente

Sobre a SubsetorID o conteúdo está na coluna B e ele deve ser a primeira célula acima da linha que contenha um valor não nulo e diferente da palavra 'SUBSETOR' ela terá o Descritivo que corresponde a coluna Descritivo

na tabela dbo.Subsetor e a partir desse Descritivo deverá trazer o id correspondente

Sobre a SegmentoID o conteúdo está na coluna C e ele deve ser a primeira célula acima da linha que contenha um valor não nulo e diferente da palavra 'SEGMENTO' e que tenha um valor nulo na coluna D

ela terá o Descritivo que corresponde a coluna Descritivo na tabela dbo.Segmento e a partir desse Descritivo deverá trazer o id correspondente

```
CREATE TABLE dbo.Empresa (
    ID INT IDENTITY(1,1) PRIMARY KEY, --
    Chave primária autoincrementada
    Nome VARCHAR(255) NOT NULL, --
    Nome da empresa
    Codigo VARCHAR(50) NOT NULL, --
    Código/Sigla da empresa

    -- Chaves estrangeiras
    SegmentoClassificacaoID INT NULL, --
    FK opcional (NÃO obrigatória)
    SetorEconomicoID INT NOT NULL, --
    FK obrigatória
    SubsetorID INT NOT NULL, --
    FK obrigatória
    SegmentoID INT NOT NULL, --
    FK obrigatória
```

```
-- Definição das chaves estrangeiras
CONSTRAINT
FK_Empresa_SegmentoClassificacao FOREIGN KEY
(SegmentoClassificacaoID)
REFERENCES
dbo.SegmentoClassificacao(ID),

CONSTRAINT FK_Empresa_SetorEconomico
FOREIGN KEY (SetorEconomicoID)
REFERENCES dbo.SetorEconomico(ID),

CONSTRAINT FK_Empresa_Subsetor FOREIGN
KEY (SubsetorID)
REFERENCES dbo.Subsetor(ID),

CONSTRAINT FK_Empresa_Segmento FOREIGN
KEY (SegmentoID)
REFERENCES dbo.Segmento(ID)
);
```

Fonte: Elaborado pelo autor (2025)

3- Consulta de dados sobre as ações

Como criar uma API em FatsAPI para retornar todos os resultados da tabela

```
CREATE TABLE dbo.SegmentoClassificacao (
    ID INT IDENTITY(1,1) PRIMARY KEY, --
    Chave primária autoincrementada
    Sigla VARCHAR(10) NOT NULL, --
    Sigla extraída dos parênteses
    Descritivo VARCHAR(255) NOT NULL --
    Texto após os parênteses
);
```

usando clean architecture?

Preciso criar uma API que retorne os dados da tabela dbo.Empresa sendo que ela precisa retornar os dados das tabelas que tenham chaves estrangeiras

```
CREATE TABLE dbo.SegmentoClassificacao (
    ID INT IDENTITY(1,1) PRIMARY KEY, --
    Chave primária autoincrementada
    Sigla VARCHAR(10) NOT NULL, --
    Sigla extraída dos parênteses
    Descritivo VARCHAR(255) NOT NULL --
    Texto após os parênteses
);
```

```
CREATE TABLE dbo.[SetorEconomico] (
    ID INT IDENTITY(1,1) PRIMARY KEY, --
Chave primária autoincrementada
    Descritivo VARCHAR(255) NOT NULL --
Nome do setor econômico
);

CREATE TABLE dbo.Subsetor (
    ID INT IDENTITY(1,1) PRIMARY KEY, --
Chave primária autoincrementada
    Descritivo VARCHAR(255) NOT NULL --
Nome do subsetor
);

CREATE TABLE dbo.Segmento (
    ID INT IDENTITY(1,1) PRIMARY KEY, --
Chave primária autoincrementada
    Descritivo VARCHAR(255) NOT NULL --
Nome do segmento
);

CREATE TABLE dbo.Empresa (
    ID INT IDENTITY(1,1) PRIMARY KEY, --
Chave primária autoincrementada
    Nome VARCHAR(255) NOT NULL, --
Nome da empresa
    Codigo VARCHAR(50) NOT NULL, --
Código/Sigla da empresa

    -- Chaves estrangeiras
    SegmentoClassificacaoID INT NULL, --
FK opcional (NÃO obrigatória)
    SetorEconomicoID INT NOT NULL, --
FK obrigatória
    SubsetorID INT NOT NULL, --
FK obrigatória
    SegmentoID INT NOT NULL, --
FK obrigatória

    -- Definição das chaves estrangeiras
CONSTRAINT
FK_Empresa_SegmentoClassificacao FOREIGN KEY
(SegmentoClassificacaoID)
REFERENCES
dbo.SegmentoClassificacao(ID),

CONSTRAINT FK_Empresa_SetorEconomico
FOREIGN KEY (SetorEconomicoID)
REFERENCES dbo.SetorEconomico(ID),
```



```
        CONSTRAINT FK_Empresa_Subsetor FOREIGN  
KEY (SubsetorID)  
        REFERENCES dbo.Subsetor(ID),  
  
        CONSTRAINT FK_Empresa_Segmento FOREIGN  
KEY (SegmentoID)  
        REFERENCES dbo.Segmento(ID)  
    );
```

seguinte o padrão:

```
def get_all_segmento(self) ->  
List[Tuple[int, str]]:  
    conn = get_db_connection()  
    cursor = conn.cursor()  
    query = "SELECT ID, Descritivo FROM  
dbo.Segmento"  
    cursor.execute(query)  
    result = cursor.fetchall()  
    conn.close()  
    return [(row.ID, row.Descritivo)  
for row in result]
```

Fonte: Elaborado pelo autor (2025)

4- Consumir dados de APIs externas

Como consumir dados do seguinte endereço
<https://api.bcb.gov.br/dados/serie/bcdata.sgs.12/dados?formato=json&dataInicial=08/04/2015&dataFinal=08/04/2025> por meio de um adapter (design pattern) em clean archtecte usando FastAPI e salvando os dados consumidos na tabela

```
CREATE TABLE CDI_Diario (  
    Data DATE NOT NULL,  
    Valor FLOAT NOT NULL,  
    PRIMARY KEY (Data)  
);
```

Como fazer uma API que consulte os dados na tabela usando filtro de data inicial e final

```
CREATE TABLE CDI_Diario (  
    Data DATE NOT NULL,  
    Valor FLOAT NOT NULL,
```

```
PRIMARY KEY (Data)
);
```

Como criar uma API para consultar os dados do IBovespa usando YFinance, por meio de um adapter (design pattern), em FastAPI e depois da consulta salvar os dados em uma tabela no banco de dados

```
CREATE TABLE IBOV_Historico (
    Data DATE NOT NULL PRIMARY KEY,
    Abertura FLOAT NOT NULL,
    Alta FLOAT NOT NULL,
    Baixa FLOAT NOT NULL,
    Fechamento FLOAT NOT NULL,
    Volume BIGINT NOT NULL
);
```

Como fazer uma API que consulte os dados na tabela usando filtro de data inicial e final

```
CREATE TABLE IBOV_Historico (
    Data DATE NOT NULL PRIMARY KEY,
    Abertura FLOAT NOT NULL,
    Alta FLOAT NOT NULL,
    Baixa FLOAT NOT NULL,
    Fechamento FLOAT NOT NULL,
    Volume BIGINT NOT NULL
);
```

Como criar uma API para consultar os dados de ações listadas na Bovespa usando YFinance, por meio de um adapter (design pattern), em FastAPI e depois da consulta salvar os dados em uma tabela no banco de dados

```
CREATE TABLE Acao_Historico (
    Data DATE NOT NULL,
   Codigo VARCHAR(10) NOT NULL,
    Abertura FLOAT NOT NULL,
    Alta FLOAT NOT NULL,
    Baixa FLOAT NOT NULL,
    Fechamento FLOAT NOT NULL,
    Volume BIGINT NOT NULL,
    PRIMARY KEY (Data, Codigo) -- Permite
    várias ações diferentes na mesma data
);
```

Preciso criar uma FastAPI usando clean architecture que retorne o índice Beta sendo que tenho duas tabelas uma com índice Bovespa e outra com o histórico das ações sendo as tabelas

```
CREATE TABLE IBOV_Historico (  
    Data DATE NOT NULL PRIMARY KEY,  
    Abertura FLOAT NOT NULL,  
    Alta FLOAT NOT NULL,  
    Baixa FLOAT NOT NULL,  
    Fechamento FLOAT NOT NULL,  
    Volume BIGINT NOT NULL  
);  
  
CREATE TABLE Acao_Historico (  
    Data DATE NOT NULL,  
    Codigo VARCHAR(10) NOT NULL,  
    Abertura FLOAT NOT NULL,  
    Alta FLOAT NOT NULL,  
    Baixa FLOAT NOT NULL,  
    Fechamento FLOAT NOT NULL,  
    Volume BIGINT NOT NULL,  
    PRIMARY KEY (Data, Codigo) -- Permite  
várias ações diferentes na mesma data  
);
```

Fonte: Elaborado pelo autor (2025)

5- Cálculo de índices

Como calcular o índice Sharpe de uma ação sendo que tenho a seguinte tabela:

```
CREATE TABLE Acao_Historico (  
    Data DATE NOT NULL,  
    Codigo VARCHAR(10) NOT NULL,  
    Abertura FLOAT NOT NULL,  
    Alta FLOAT NOT NULL,  
    Baixa FLOAT NOT NULL,  
    Fechamento FLOAT NOT NULL,  
    Volume BIGINT NOT NULL,  
    PRIMARY KEY (Data, Codigo) -- Permite  
várias ações diferentes na mesma data  
);
```

Como calcular a volatilidade de uma ação usando a FastAPI sendo que tenho a seguinte tabela:

```
CREATE TABLE Acao_Historico (  
    Data DATE NOT NULL,  
    Codigo VARCHAR(10) NOT NULL,  
    Abertura FLOAT NOT NULL,
```

```
Alta FLOAT NOT NULL,  
Baixa FLOAT NOT NULL,  
Fechamento FLOAT NOT NULL,  
Volume BIGINT NOT NULL,  
PRIMARY KEY (Data, Codigo) -- Permite  
várias ações diferentes na mesma data  
);
```

Como calcular o retorno esperado de uma ação usando a FastAPI sendo que tenho a seguinte tabela:

```
CREATE TABLE Acao_Historico (  
    Data DATE NOT NULL,  
    Codigo VARCHAR(10) NOT NULL,  
    Abertura FLOAT NOT NULL,  
    Alta FLOAT NOT NULL,  
    Baixa FLOAT NOT NULL,  
    Fechamento FLOAT NOT NULL,  
    Volume BIGINT NOT NULL,  
    PRIMARY KEY (Data, Codigo) -- Permite  
várias ações diferentes na mesma data  
);
```

Como usar o percentil (função do Excel) no python sendo que tenho os dados de um mês de uma determinada ação?

Como selecionar os últimos dois registro da tupla def
get_ibov_data_by_date_range(self, start_date: str, end_date: str) -> List[Tuple[str, float, float,
float, float, int]]:

```
historico_ibov = self.repository.get_ibov_data_by_date_range(data_inicio, data_fim)
```

Fonte: Elaborado pelo autor (2025)